

哈希表

1. 两数之和

给定一个整数数组 `nums` 和一个整数目标值 `target`，请你在该数组中找出和为目标值 `target` 的那两个整数，并返回它们的数组下标。

你可以假设每种输入只会对应一个答案，并且你不能使用两次相同的元素。

你可以按任意顺序返回答案。

```
# LeetcodeHot100:1.两数之和
from typing import List

class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        mp = {}
        for i, num in enumerate(nums):
            if target - num in mp:
                return [mp[target - num], i]
            mp[num] = i
        return []
```

如果我们按顺序将数组一个个遍历过去，虽然也能得到正确结果但是如果数据量很大有大量重复数据，那么就有可能运行超时。

那么我们可以用哈希表来解决这个问题，如果某个数在哈希表中已经存在，那么我们就可以返回结果，如果某个数在哈希表中不存在，那么我们就将这个数存入哈希表。

tips1: 这里 `enumerate()` 函数可以返回数组的索引和值，等价于 `i` 和 `nums[i]`

tips2: `mp={}` 创建字典，`mp[x]=y` 将 `x` 作为键，`[y]` 作为值存入 `mp` 字典中，当进行搜索时，我们得到相同结果就会直接查到 `mp[x]`，然后返回 `[y]`，避免空间重复创建

那么哈希表适合用于什么情况呢？

总结: 当我们需要遍历数组或者搜索数据的时候结果出现大量重复的答案时，哈希表就能够避免多余搜索快速返回答案

2. 字母异位词分组

给你一个字符串数组，请你将字母异位词组合在一起。可以按任意顺序返回结果列表。

示例 1:

- 输入: `strs = ["eat", "tea", "tan", "ate", "nat", "bat"]`
- 输出: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

解释:

- 在 `strs` 中没有字符串可以通过重新排列来形成 "bat"。
- 字符串 "nat" 和 "tan" 是字母异位词，因为它们可以重新排列以形成彼此。
- 字符串 "ate", "eat" 和 "tea" 是字母异位词，因为它们可以重新排列以形成彼此。

示例 2:

- 输入: `strs = [""]`
- 输出: `[[""]]`

示例 3:

- 输入: `strs = ["a"]`
- 输出: `[["a"]]`

提示:

- $1 \leq \text{strs.length} \leq 10^4$
- $0 \leq \text{strs}[i].\text{length} \leq 100$
- `strs[i]` 仅包含小写字母

我们来简单分析下这题的思路。

我们要将字符串和字符串匹配，通常会尝试用二重循环枚举所有位置两两组合的所有情况。

那么如果要匹配这两个字符串是否是异位词，就需要尝试将他们变成相同的排序，然后比较结果是否相同，比如 `eat` 和 `tea`，都会变成 `aet` 和 `aet`，那么他们就相等。

这样我们通过二重循环马上就能得到答案。

但是这样时间复杂度太高，对于数据高度的输入，就会可能超时。

我们不难发现将单个字符串排序后我们可以存入哈希表建立快速匹配。

这样我们只需要遍历一次数组，将里面的字符串取出来排序存入哈希表，然后遍历哈希表，将值取出来返回，这样时间复杂度就降到 $O(n)$ 绝对不会超时了。

```
# LeetcodeHot100:2.字母异位词分组
from typing import List
from collections import defaultdict

class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        mp = defaultdict(list)
        for s in strs:
            k = ''.join(sorted(s))
            mp[k].append(s)
        res = list(mp.values())
        return res
```

tips1: `defaultdict(list)` 创建一个字典，如果字典中不存在这个键，那么就会创建一个空列表，并返回这个列表。这个也可用 `mp={}` 创建字典，但是这样字典中不存在的键会返回 `None`，需要 `mp.get(k, [])` 设置默认值

tips2: `''.join()` 函数可以将列表中的元素连接成一个字符串，默认连接符为空

tips3: `sorted()` 函数将列表中的元素排序并返回一个新的列表，默认排序规则为升序，如果需要降序，可以加参数 `reverse=True`

tips4: `list()` 函数将字典的值转换为列表，返回一个新的列表

3. 最长连续序列

相关企业

给定一个未排序的整数数组 `[nums]`，找出数字连续的最长序列（不要求序列元素在原数组中连续）的长度。

请你设计并实现时间复杂度为 $O(n)$ 的算法解决此问题。

示例 1:

- 输入: `nums = [100,4,200,1,3,2]`
- 输出: `4`
- 解释: 最长数字连续序列是 `[1, 2, 3, 4]`。它的长度为 4。

示例 2:

- 输入: `nums = [0,3,7,2,5,8,4,6,0,1]`
- 输出: 9

示例 3:

- 输入: `nums = [1,0,1,2]`
- 输出: 3

提示:

- $0 \leq \text{nums.length} \leq 10^5$
- $-10^9 \leq \text{nums}[i] \leq 10^9$

分析下思路，这题需要我们找最长的数字连续序列，我们不难想到直接给它排序一下，然后遍历数组，如果后一个数减去前一个数等于 1，那么就更新最长连续序列的长度。

那么我们可以开一个 `set` 集合，这样就不用考虑去重了，比如出现 `[1,1,2,3]` 这种情况。我们需要去重减少点遍历次数。

1. 遍历 `set` 数组，将取出的元素进行判断
2. 因为遍历是 `set` 数组，我们只需要对当前循环中的 `num`，+1 或者 -1 判断是否也在 `set` 数组中。如果存在那么更新长度，因为需要统计最长的长度所以我们需要两个变量去存

```
class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        if len(nums) == 0:
            return 0
        num_set = set(nums)
        max_l = 0
        for num in num_set:
            current = num
            cur_l = 1
            while current+1 in num_set:
                current += 1
                cur_l += 1
            max_l = max(max_l, cur_l)
        return max_l
```

这样可以通过大部分样例但是不是最优解，因为会超时。

因为我们忘记考虑 $num-1$ 了，如果 $num-1$ 在 num_set 中，那么 $num-1+1$ 一定在 num_set 中，那么我们就需要判断 $num-1+1$ 了。

所以只需要在这个循环中判断 $num-1$ 是否在 num_set 中就可以优化了。

LeetcodeHot100:3.最长连续序列

```
class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        if len(nums) == 0:
            return 0
        num_set = set(nums)
        max_l = 0
        for num in num_set:
            if num - 1 not in num_set:
                current_num = num
                current_l = 1
                while current_num + 1 in num_set:
                    current_num += 1
                    current_l += 1
                max_l = max(max_l, current_l)
        return max_l
```

那还有没有更简洁的方法呢？

我们其实还可以进一步提高效率和空间效率，思路是什么呢？

就是我们直接判断前后两个数字是否连续，然后依然用两个数记录长度一个数记录当前最长长度，一个记录所有最大长度。

那么如果连续就更新当前长度，如果不连续计算所有最大长度，然后重置当前长度。

这个思路是不是比前面的简单的多的多呢。

```

class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        if len(nums) == 0:
            return 0
        nums = sorted(nums)
        max_l = cur_l = 1
        for i in range(1, len(nums)):
            if nums[i] == nums[i-1] + 1:
                cur_l += 1
            elif nums[i] != nums[i-1]: # 跳过重复元素
                max_l = max(max_l, cur_l)
                cur_l = 1
        return max(max_l, cur_l)

```

因为我们给出的这个思路用了排序，题目要求时间复杂度为 $O(n)$ 。

我们排序了就是 $O(n \log n)$ ，但是我们想到这个思路就是为了进一步找到最优的方法那么我们接下来去掉排序就会发现更简单了！

直接利用关系，如果当前数字减去 1 不在集合中，那么当前数字就是连续序列的开始。

```

class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        st = set(nums)
        ans = 0
        for num in st:
            if num-1 in st:
                continue
            nxt = num+1
            while nxt in st:
                nxt += 1
            ans = max(ans, nxt-num)
        return ans

```